HERMES PLUGIN HACKATHON · VERIFICATION DOSSIER

Peripheral

A memory of what was on your screen — hands-on verification, claim by claim.

Version 2.1.1 · catalog pins the commit tagged `v2.1.1` (`scripts/check_release.py` proves `git rev-parse v2.1.1` equals the `sha` in `catalog-entry.yaml`) · docs built at `64c37ff0af9e` (master) — the tree immediately before this file was committed, since a PDF cannot contain the SHA of the commit that adds it — the tag `v2.1.1` names the commit that does · 2026-11-05 19:35 UTC · MIT · `okyssta/hermes-peripheral`

> **The claim.** Hermes can look at the screen right now; it cannot answer for a screen that is already gone. Peripheral captures the screen itself, reads it locally, stores it with the time it appeared, and gives the agent six tools to search it, open it — as text and as the image — and forget it. No cloud, no account, no API key for indexing.

1. What ships

SURFACE	WHAT IT IS
`capture.py`	capture backends, argv construction, timeouts
`ingest.py`	scan, change gate, capture-and-ingest
`db.py`	SQLite/FTS5, forget (rows + files), prefix fallback
`tools.py`	six tool handlers, multimodal envelope
`commands.py`	/peripheral surface
`schemas.py`	tool schemas the model sees
`settings.py`	config resolution
`render.py`	results, timeline, status
`skills/peripheral`	bundled skill: when to reach for screen memory, and the honesty rules for quoting OCR

Tools the model sees

TOOL	BEHAVIOUR
`screen_capture(note, image)`	photograph the display now, index it, hand it back; reports the backend and timing
`screen_recall(query, since, until, limit)`	BM25 over OCR text, re-ranked by recency and richness; prefix fallback for OCR noise
`screen_timeline(since, until, limit)`	screens and the agent's own tool calls interleaved on one clock
`screen_show(id, chars, image)`	one screen's full text, plus the image attached when the model accepts images

`screen_index(paths, batch)`	index explicit files/folders, bounded batch
`screen_forget(id, all)`	delete rows, thumbnails and captured frames; never the user's own screenshots

2. Claim ledger

Each line says what kind of evidence backs it. Nothing in this document is upgraded past its label.

CLAIM	STATUS	EVIDENCE
The plugin captures a real display and indexes what it captured	**DEMONSTRATED**	three real browser pages on Xvfb :99, captured via `ffmpeg x11grab` in 1.2 s / 0.9 s / 1.5 s, 679 / 465 / 428 characters of OCR text; frames are in the demo video and re-generated by `scripts/demo_capture.py`
The model chooses the tools itself and answers from the memory	**DEMONSTRATED**	6 tool calls, 18.79 s, model `deepseek-v4.1-flash`; full transcript committed at `docs/demo-transcript.txt`
Screens survive the window closing; a text query finds them	**DEMONSTRATED**	the demo's first screen was captured, then the browser was closed and replaced twice; `screen_recall("card declined")` still returned it, and all three screens matched a single question
Forget removes the pixels, not just the row	**DEMONSTRATED**	`test_forget_deletes_a_captured_frame_and_spares_a_user_screenshot`, `test_forget_all_clears_captured_frames_and_keeps_user_files`; live probe in the transcript of the forget run
A locked index cannot stall a turn	**DEMONSTRATED**	measured with an exclusive lock held by a second connection: five `post_tool_call` events took **25.0 s** before 2.1.1 (Hermes' 5 s busy timeout, inherited), **7.5 s** with only that timeout capped, and **3 ms** once the event log stopped waiting at all — the row is dropped and the tool call proceeds (`tests/test_locked_db.py`). Reads and the timeline kept answering throughout, because WAL lets readers through

Works without Tesseract (visual index)	**DEMONSTRATED**	`test_a_machine_without_ocr_still_builds_a_visual_index` — image stored, timeline lists it, marked `no text`
No network calls, no telemetry	**DEMONSTRATED**	no HTTP client imported anywhere in the plugin; `hermes plugins validate` security scan: safe; only local subprocesses (tesseract, capture binary)
Useful to ~90% of Hermes users	**HYPOTHESIS**	true for anyone with a display (capture works with no screenshot habit) or a screenshot folder; a headless user with neither gets nothing until they add a folder — stated, not hidden
Native image attachment in tool results	**DEMONSTRATED**	envelope shape asserted by `test_screen_show_attaches_the_image_when_the_model_can_see`; the gate by `tests/test_vision_gate.py` (explicit `model.supports_vision`, `agent.image_input_mode`, per-model `providers.<p>.models.<m>.supports_vision`, non-bool values ignored, unreadable config fails closed). With `model: gpt-4o / provider: openai` plus a per-model override, `screen_show` returns the `_multimodal` envelope with the image; with a text-only model it stays a string — both executed here. The plugin agrees with the host's own gate on 4 of 5 probes: the one it declines is a model the models.dev catalogue knows and `config.yaml` does not, because that lookup is a private core function and rule 9 asks plugins to stay public. `image: true` covers that case
A live end-to-end agent turn through the Hermes gateway	**NEGATIVE**	`hermes chat -q` on this host rejects new sessions with `invalid_params` for any prompt, with the plugin disabled and on a second profile — independent of Peripheral. The demo therefore drives the same handlers through the provider's tool-calling API and commits the transcript
Capture on a headless server	**NEGATIVE**	there is no display to photograph; `screen_capture` returns the reason and `/peripheral status` reports no backend. The screenshot index still works
Cross-platform capture (Wayland, macOS, Windows)	**IMPLEMENTED, UNVERIFIED HERE**	argv construction is unit-tested (`test_x11_capture_is_an_argv_list_and_never_a_shell_string`, `test_missing_binary_means_no_command`); only the X11/ffmpeg path could be executed on this machine

3. The session behind the demo

Question: I got a card error a little while ago — what did it actually say? And was anyone else talking about it?

TOOL	ARGUMENTS CHOSEN BY THE MODEL	RESULT CHARS

screen_recall	{"query": "card declined", "since": "today", "limit": 15}	1,308
screen_recall	{"query": "card error", "limit": 15}	933
screen_recall	{"query": "payment failed", "limit": 15}	957
screen_show	{"id": 1}	1,091
screen_show	{"id": 2}	881
screen_show	{"id": 3}	841

Index at the time: 3 screens (3 captured live), 1,572 chars of OCR text · capture: ready (x11-ffmpeg) · captured 2026-11-05T17:07:14Z

The answer it produced (verbatim, trimmed here for the page)

> Here's what was on screen: **The error itself** (17:06, a Stripe checkout page in TEST MODE): > **Payment failed** — The charge for invoice INV-2048 was not completed. > **Your card was declined** — The card has expired. Check the expiration date or use a different card. > `Error code: expired card - decline_code: expired card` Details on the page: customer Acme Robotics Ltd, invoice INV-2048, card 4242 ending, **expires 03/24**, amount **$1,240.00 USD**. The page noted Stripe retries the charge three times over the next week, and if the card is replaced the retry succeeds automatically. **Yes — two people were talking about it in #deploys** (17:07): - **Sam Okafor:** "Heads up — the checkout service is throwing expired card for test cards with old expiry dates. Not a real customer issue, but it breaks the smoke test." - **Priya Raman:** "That's Stripe's sandbox behaviour, the card fixture is pinned to 03/24. I'll rotate the fixture after lunch." Then deploy-bot reported `checkout-api@a91f3c2` going to staging, followed by: **"Smoke test failed: payment_flow_test — expected 200, got 402 (card_declined)."** **One thing worth flagging:** the thread frames this as a sandbox-only fixture problem, but the production dashboard you also had open showed **402 card_declined: 18,402** errors in the last 24 hours on `/v1/checkout/pay`, with the error rate at 1.9% (+0.7 pts, labelle

4. Verification performed

TESTS

64 passed in 10.3 s — real PNGs, real Tesseract, real SQLite, no mocks except the capture binary seam.

MANIFEST

`hermes plugins validate .` → **Validation passed**; security scan **safe**; no core override; declared tools and hooks match registrations.

DEMO

80.0 s, silent, 1600x1000 h264, 0.9 MB — under the 90 s cap.

LOCALITY

one SQLite file + WebP files under `plugin-data/peripheral/`; no socket, no HTTP, no key.

Measured on this machine

OPERATION	TIME	NOTES
Capture one 1600x1000 frame (X11, ffmpeg)	116 ms warm / 1.2 s cold	includes process start
OCR that frame (Tesseract 5.3.4, 1 thread)	654 ms	679 characters read
Index 10 screenshots at 1440x900	10.6 s	12,616 characters, 1.3 MB source

Warm rescan of the same folder	0.00 s	0 images re-read
Model turn: 6 tool calls + answer	18.8 s	deepseek-v4.1-flash, provider-side latency included

5. Privacy model

PROPERTY	HOW IT IS ENFORCED
Capture is opt-in	`capture_enabled` defaults to `false`; the background watcher is not even constructed unless it is true
Only changed frames are stored	dHash distance gate in `ingest.capture_and_ingest(force=False)`; a static desktop writes nothing
Who can start a capture	the model, by calling `screen_capture` — a normal tool call, visible in the session and written to the event log; or the user, through `/peripheral capture`. Background capture is separate and starts only when the user enables it
Background capture rate	`capture_max_per_hour` (120) and `capture_min_gap_seconds` (15) bound how fast frames accumulate — a rate limit, not a size limit
Retention is opt-in	`retention_days` defaults to `0` (keep everything); when set, every scan deletes older rows and their thumbnails and frames. Disk use is rate-bounded by default, size-bounded only once retention or forget is used
Redaction at the storage boundary	`db.record_event` and the OCR path apply `ocr.redact` before insert; test asserts a password never reaches the DB
Forget means forget	`db.forget` unlinks thumbnails *and* captured frames, outside the lock, after the commit; the user's own files are never unlinked
The agent can be told to forget	`screen_forget` tool, advertised in its schema
No shell injection surface	every capture command is an argv list; no `shell=True` anywhere

6. Reproduction

```
git clone https://github.com/okyssta/hermes-peripheral && cd hermes-peripheral
uv venv /tmp/peripheral-venv && uv pip install --python /tmp/peripheral-venv/bin/python -r requirements-dev.txt
/tmp/peripheral-venv/bin/python -m pytest tests -q   # 64 passed
hermes plugins validate .                    # Validation passed

# rebuild the demo evidence from scratch (needs a display + a browser)
Xvfb :99 -screen 0 1600x1000x24 &
DISPLAY=:99 python3 scripts/demo_capture.py
HERMES_HOME=/tmp/demo python3 scripts/demo_session.py "what was that card error?"
```

```
python3 scripts/make_demo_video.py
python3 scripts/make_dossier.py
```

Artifact hashes

FILE	SHA256
demo.mp4	cf6552582c0e17abefa2da019f3e10410aab6d4c24567d4103b8391cb707cce6
demo-transcript.json	9b1ac0c8b99c5099c969867cf5b0d73fbbbf4f9b1e3699889f025e6aae20f65b
demo-transcript.txt	3fe187adae7b4838f6d84fcfa81e1a19db58ef721edcdb6c7492a5c39ee8b0e4

7. Limits and open risks

- **OCR noise is real.** The demo's own OCR read `expired_card` as `expired card` and "Acme Robotics Ltd" as "Acme Robolies Lid". Mitigations: prefix fallback in search, snippet re-ranking, and the image attachment so the model can read what OCR could not.
- **Background capture sees everything on the display.** That is the point and the risk. It is off by default, rate-limited, and deletable per screen; the README says so in the privacy section rather than burying it.
- **Native image attachment follows the config, not the catalogue.** A screen is attached when `config.yaml` says the model can see (`model.supports_vision`, `agent.image_input_mode`, or a per-model `providers.<p>.models.<m>` override); a model the models.dev catalogue knows but the config does not falls back to text and asks for the image with `image: true`. The catalogue lookup lives behind a private core function, and catalog rule 9 asks plugins to stay on public surfaces — so this is a stated trade, not an oversight
- **Cross-platform capture paths are implemented, argued and unit-tested, but only X11 was executed here.** A maintainer on macOS/Windows would be the right person to confirm the other backends.
- **No live gateway turn was possible on this host** (`invalid_params`, plugin-independent). The transcript is a real provider session against the real handlers, not a rendered script.

8. Change log: 1.0.1 → 2.1.1

AREA	CHANGE
Capture	new `capture.py`: X11 (ffmpeg), Wayland (grim), macOS (screencapture), Windows (PowerShell), ImageMagick/scrot/maim fallbacks; argv-only, timeouts, no shell
Tools	`screen_capture` and `screen_forget` added; `screen_show` returns the screen itself on vision models
Indexing	captured frames stored as WebP; visual index when Tesseract is absent; change gate for background capture
Privacy	forget and retention now delete captured frames as well as thumbnails; captured frames live only under plugin-data
Search	prefix fallback for OCR noise, with an honest-miss test
Surface	`/peripheral capture`, `/peripheral watch on|off`, capture readiness in status; skill rewritten

Tests	27 → 64
Vision (2.1.0)	the gate now reads the documented config keys Hermes itself reads — `model.supports_vision`, `agent.image_input_mode`, and a per-model `providers.<p>.models.<m>.supports_vision` override — instead of only the top-level flag, and a quoted string is ignored rather than read as a boolean. It still imports no Hermes internals (rule 9); a catalogue-only vision model falls back to text and the model asks with `image: true`
Dev deps (2.1.0)	`requirements-dev.txt` declares the test-only packages (pytest, Pillow, PyYAML); a fresh clone can reproduce the suite without being told which module the harness imports
Locked index (2.1.1)	`db.record_event` — the write that `post_tool_call` makes after every tool call — no longer queues behind a lock another process holds: it attempts the insert with no busy wait, drops the row if the index is busy, and restores the 1.5 s cap for everything else. Measured under an exclusive lock: 25.0 s of stall across five tool events before, 3 ms after
Docs	2.0.2: the dossier names the release tag the catalog pins instead of a build-time SHA (the value that drifted twice), and `scripts/check_release.py` now proves tag == pin and that the shipped PDF is the recorded build; the catalog disclosure says the model can call `screen_capture` and that the call is visible; "bounded disk" is corrected to a capture-*rate* bound, with retention opt-in